

RxAide: An AI-Powered Medication Reminder and Management Assistant

Aminul Islam
CSE299, Section 4, Group 2
NSU ID: 2321169042

Md Shahriar Rakib Rabbi
CSE299, Section 4, Group 2
NSU ID: 2321937642

Tirtho Mojumdar
CSE299, Section 4, Group 2
NSU ID: 2312536042

Abstract—Medication non-adherence is a pervasive healthcare challenge that contributes to approximately 125,000 preventable deaths annually in the United States alone, and generates an estimated \$100–289 billion in avoidable healthcare costs worldwide. This paper presents RxAide, a native Android application that addresses three critical gaps identified in existing medication management solutions: the *Integration Gap* (no unified pipeline from prescription scanning to adherence tracking), the *Accessibility Gap* (manual data-entry burdens on elderly and polypharmacy patients), and the *Intelligence Gap* (absence of context-aware conversational assistance). RxAide combines AI-powered prescription scanning via the Google Gemini API with a novel three-pass extraction and multi-candidate verification pipeline, automated schedule generation, reliable WorkManager-based reminders, dose adherence tracking, and a conversational AI assistant—capabilities that no single existing application offers. The system employs a carefully engineered prompt architecture supporting bilingual Bangla/English prescriptions and Bangladesh-specific dosage notation. Built with Kotlin, Jetpack Compose, and Room database following the MVVM architecture pattern, the application was developed iteratively over five weeks with 37 commits across multiple branches. This report details the design, implementation, and evaluation of RxAide, demonstrating a functional end-to-end medication management system that bridges the identified gaps in the current application landscape.

Index Terms—medication adherence, Android application, AI prescription scanning, Google Gemini API, WorkManager, MVVM, mHealth, Bangla prescription, automated scheduling

I. INTRODUCTION

A. Background and Motivation

Medication adherence—defined as the extent to which patients take medications as prescribed—is critical for effective treatment outcomes. The World Health Organization reports that adherence to long-term therapies averages only 50% in developed countries, with significantly lower rates in developing nations [1]. This non-adherence results in approximately 125,000 preventable deaths annually in the United States alone [2], [22] and accounts for 10% of all hospital admissions [3], [22]. The economic burden is equally severe, generating an estimated \$100–289 billion in avoidable healthcare costs each year [2], [4], [22].

Elderly patients managing polypharmacy regimens—often five or more concurrent medications—face disproportionately elevated risk [1]. Complex dosing schedules, varying frequencies, and multiple timing requirements create a cognitive burden that frequently leads to missed or incorrect doses.

Mobile health (mHealth) applications have emerged as a promising intervention strategy. A 2025 systematic review and meta-analysis by Kim et al., encompassing 26 randomized controlled trials (RCTs) and 5,174 participants, demonstrated that mobile applications have a statistically significant positive impact on medication adherence (OR = 2.371, SMD = 0.279), with the greatest effectiveness observed when using interactive strategies (OR = 2.652) and advanced reminders (OR = 1.849) [5]. An additional systematic review by Lanke et al. covering 14 RCTs published between 2014 and 2022 corroborated these findings, with 10 of the 14 studies showing statistically significant improvements in adherence [6].

B. Problem Definition

Despite the proven efficacy of mobile interventions, a comprehensive review of seven existing medication management applications (detailed in Section II) reveals three critical gaps in the current landscape.

- 1) **The Integration Gap:** No existing application provides a unified, end-to-end pipeline from prescription image capture through data extraction, automatic schedule generation, reminders, and adherence tracking. Users are forced to either enter all data manually or combine multiple separate tools.
- 2) **The Accessibility Gap:** Manual data entry—required by every major reminder application including Medisafe [7] and MyTherapy [8]—disproportionately burdens elderly patients, those with limited technical literacy, and individuals managing complex multi-drug regimens.
- 3) **The Intelligence Gap:** While basic timer-based reminders are abundant, context-aware conversational assistance that understands a user’s medication context and can handle schedule changes through natural language remains virtually absent.

C. Contributions

This paper presents RxAide, an Android application designed to address all three identified gaps through an integrated, AI-powered medication management experience. The key contributions are:

- A novel three-pass prescription extraction pipeline with multi-candidate medicine name verification using Google Search grounding, supporting bilingual Bangla/English prescriptions and Bangladesh-specific dosage notation.

- The first implementation of automated schedule generation from prescription data—a capability entirely absent from every existing application reviewed.
- A context-aware conversational AI assistant capable of executing real database modifications through structured action blocks, enabling natural-language medication management.
- A complete, functional end-to-end system integrating prescription scanning, schedule generation, reliable reminders, dose tracking, and conversational assistance in a single native Android application.

II. LITERATURE REVIEW

A comprehensive review of seven existing commercial applications was conducted to identify the current state of the art and gaps in medication management technology. Applications were selected based on their relevance to the core features of RxAide: prescription scanning, medication reminders, adherence tracking, and AI-powered assistance.

A. Existing Applications

Smart Meds Reminder [9] provides customizable scheduling and notification capabilities with dose logging and adherence tracking. While its premium tier now includes OCR-based scanning of medication packaging, it does not offer end-to-end handwritten prescription scanning, automated schedule generation, or conversational AI features.

Prescription Reader AI [10] focuses on leveraging AI and OCR technology to scan handwritten prescriptions and extract medication details. However, it is a single-purpose digitization tool: it offers no medication reminders, dose tracking, adherence monitoring, or schedule generation from extracted data.

AI Prescription Saathi [11] similarly employs AI-based OCR for prescription digitization and, as of 2026, also advertises basic reminders and prescription organization features. Nevertheless, it does not provide an automated schedule generation pipeline or a context-aware conversational AI assistant.

Medisafe [7] is among the most established reminder applications worldwide, offering highly customizable reminders, drug interaction warnings, caregiver support, and comprehensive adherence reports. Despite its extensive feature set, it lacks AI-powered prescription scanning and requires manual medication entry.

MyTherapy [8] combines medication reminders with broader health monitoring, including barcode scanning for adding medications from packaging, but does not support handwritten prescription scanning or conversational AI features.

CareZone [12] offers pill bottle label scanning and pharmacy integration but does not support handwritten prescription scanning. Its future development remains uncertain following Walmart’s acquisition of its medication management technology platform in 2020 [23].

DoseMed [13] is the only reviewed application integrating a conversational AI assistant, providing real-time medication

support. However, it relies on manual medication entry with no prescription scanning capability, and does not offer automated schedule generation.

B. Comparative Analysis

Table I presents a systematic feature comparison across all reviewed applications and RxAide.

TABLE I
FEATURE COMPARISON OF EXISTING APPLICATIONS VS. RXAIDE

Capability	SmartMeds	RxReader	Medisafe	MyTherapy	DoseMed	RxAide
AI Presc. OCR	✗	✓	✗	✗	✗	✓
Auto Schedule	✗	✗	✗	✗	✗	✓
Reminders	✓	✗	✓	✓	✓	✓
Dose Tracking	✓	✗	✓	✓	✓	✓
Conv. AI	✗	✗	✗	✗	✓	✓

C. Key Findings

The most significant finding is the **absence of automated schedule generation**: every cell in the “Auto Schedule Generation” row is ✗ except RxAide. This represents the most critical unmet need—no existing application automatically converts prescription data into a ready-to-use medication schedule. Additionally, the market exhibits a clear dichotomy: applications either excel at AI-powered prescription digitization or at medication reminding and adherence tracking, but no existing solution effectively bridges both domains.

D. Technology Landscape

Recent advances in multimodal AI, particularly the Google Gemini API [14], enable simultaneous processing of prescription images and contextual medical knowledge within a single inference call. Google has developed Med-Gemini, a family of specialized models achieving 91.1% accuracy on MedQA (USMLE) benchmarks [15], demonstrating the platform’s capability for healthcare applications. In the domain of handwritten prescription recognition, deep learning approaches including CNN-LSTM hybrid architectures [16] and multi-head attention models [17] have shown promising results for medicine name extraction from handwritten text.

III. SYSTEM ARCHITECTURE AND DESIGN

A. Architecture Overview

RxAide follows the Model-View-ViewModel (MVVM) architecture pattern [18], providing clean separation between the user interface, business logic, and data layers. The system is organized into three distinct layers.

View Layer (Jetpack Compose). The presentation layer comprises ten composable screens built entirely with Jetpack Compose [19]—the declarative UI toolkit for Android. The application uses zero XML layouts; every UI element is defined as a composable function. Screens observe `StateFlow` emissions from ViewModels and recompose automatically

when data changes. Navigation uses sealed-class routes with animated directional slide transitions across five bottom navigation tabs (Home, Medications, Chat, Alerts, Tracker) and five push-navigation destinations.

ViewModel Layer (Business Logic). Three `AndroidViewModels` manage all business logic: `MedicationViewModel` handles medication and schedule CRUD operations, `ChatViewModel` orchestrates the entire prescription-to-scheduling pipeline and action block parsing, and `AdherenceViewModel` provides time-filtered adherence aggregation. ViewModels use `viewModelScope` for coroutine lifecycle management and expose state exclusively via `Kotlin StateFlow`.

Model Layer (Data and Services). This layer comprises Room database [20] for local data persistence, `WorkManager` [21] for reliable background notification scheduling, and the Gemini API [14] for OCR processing and conversational AI. Two repository classes abstract data access operations.

B. Database Schema

The Room database defines four entities with foreign key relationships.

- 1) **Medication** — The central entity with 15 columns: name, dosage, dosage unit, form, frequency, meal relation, start/end dates, duration, instructions, notes, custom notification sound URI, prescription image URI, and active status.
- 2) **Schedule** — Foreign key to `Medication` with CASCADE delete. Each row represents one reminder time (hour, minute, day-of-week bitmask). A medication may have multiple schedules (e.g., morning, noon, night).
- 3) **DoseHistory** — Foreign key to both `Medication` and `Schedule` with SET_NULL on delete, preserving historical records after medication deletion. Status is one of: "unmarked", "taken", or "missed".
- 4) **ChatMessage** — Standalone entity for chat persistence, storing message content, optional image URI, sender flag, and timestamp.

All Data Access Objects (DAOs) return `Kotlin Flow<T>` for reactive queries, enabling automatic UI updates when underlying data changes.

C. Technology Stack

Table II summarizes the technology stack employed.

D. End-to-End Pipeline

The core innovation of RxAide is the seamless five-stage pipeline addressing the Integration Gap.

- 1) **Capture** — User photographs their prescription via `CameraX` or selects from gallery. A confirmation screen allows retaking before submission.
- 2) **Extract** — Image bytes are sent to Gemini 2.5 Flash with a three-pass analysis prompt and Google Search grounding (Section IV).
- 3) **Confirm** — Extracted medication data is displayed as a formatted markdown summary in chat. The user reviews,

TABLE II
RXAIDE TECHNOLOGY STACK

Component	Technology	Justification
Platform	Android SDK 36 (min 26)	Notification channels need API 26+
Language	Kotlin	Null-safe, coroutine support
UI	Jetpack Compose	Declarative, less boilerplate
Database	Room	ORM with reactive Flow queries
AI Engine	Gemini 2.5 Flash	Multimodal + Search grounding
Background	WorkManager	Survives app kills & reboots
Camera	CameraX	Lifecycle-aware, Compose-ready
Navigation	Navigation Compose	Type-safe sealed-class routes
Image	Coil	Lightweight async loading
Markdown	Compose Markdown	Renders AI formatted responses
Build	Gradle + KSP	Version catalog, compile-time gen

requests corrections via natural language, and confirms accuracy.

- 4) **Schedule** — Upon confirmation, `ChatViewModel` requests structured JSON from Gemini, creates `Medication` and `Schedule` entities in Room, and enqueues `WorkManager` periodic tasks for each reminder time.
- 5) **Track** — Notifications fire at scheduled times with in-notification action buttons. The adherence dashboard aggregates taken, missed, and unmarked dose statistics with daily, weekly, monthly, and all-time filters.

IV. IMPLEMENTATION DETAILS

A. AI-Powered Prescription Extraction

The prescription extraction system is the most technically demanding component of RxAide, requiring careful prompt engineering to achieve reliable results from the inherently noisy input of handwritten prescriptions. The implementation leverages Google Gemini 2.5 Flash as a multimodal AI engine, processing both the prescription image and a detailed extraction prompt in a single inference call.

1) **Three-Pass Extraction Pipeline:** Rather than relying on a single scan, the system instructs Gemini to perform a rigorous three-pass analysis of each prescription image.

Pass 1 — Full Scan. The model examines the entire prescription from top to bottom, left to right, including margins and corners. This pass identifies every medication entry, dosage notation, and duration marking, with special attention to Bangla text since Bangladeshi doctors frequently write durations in Bengali script.

Pass 2 — Detailed Extraction. The model processes each medication line individually, reading medicine names character by character for handwritten text. For each medicine name, the model generates 2–3 possible spelling variants if the handwriting is ambiguous, then searches each candidate against

pharmaceutical databases to identify the correct spelling. The model also extracts dosage strength, form, schedule notation, duration, meal relation, and special instructions.

Pass 3 — Cross-Verification. The model counts the total medications identified and re-checks against the image to ensure completeness. Every medicine name is verified via search, and all medicines are evaluated together for clinical coherence.

This three-pass approach with a thinking budget of 24,576 tokens enables thorough handwriting analysis that significantly improves extraction accuracy compared to a single-pass approach.

2) *Multi-Candidate Medicine Name Verification:* Handwritten medicine names present a critical challenge: a single misread character can produce a completely different or non-existent medication. RxAide addresses this through a multi-candidate search verification process.

- 1) **Candidate Generation:** For any ambiguous handwriting, the model generates 2–3 possible spelling variants based on pen stroke analysis.
- 2) **Web Search Grounding:** Each candidate is searched using Google Search grounding with the query pattern “[candidate] Bangladesh medicine” to check if it corresponds to a real, commercially available medication.
- 3) **Contextual Selection:** If multiple candidates return valid results, the system selects the spelling whose therapeutic use best matches the context of the other medications in the prescription.
- 4) **Hard Constraint:** The system is explicitly instructed never to output a medicine name that returns zero search results. If verification fails, it corrects to the closest real medicine and flags the uncertainty with a warning symbol.

3) *Bangladesh-Specific Localization:* The system prompt includes comprehensive rules for decoding Bangladesh-style dosage notation, where doses are specified as (Morning + Noon + Night) patterns:

- $1+0+1 \rightarrow$ Twice daily (morning and night)
- $\frac{1}{2}+0+\frac{1}{2} \rightarrow$ Half-dose twice daily

A complete Bangla duration recognition table is included, mapping Bengali text to English durations. The system outputs all medicine names in English while translating Bangla instructions.

4) *Prompt Architecture:* The AI system uses three distinct prompts, each optimized for its specific task.

- **System Prompt (≈ 220 lines):** Defines RxAide’s identity, core behavioral rules, field definitions matching the app’s `Medication` entity, output formatting requirements, scheduling flow rules, and the complete action block specification.
- **Image Analysis Prompt:** Contains the three-pass extraction instructions and multi-candidate verification rules, sent alongside the prescription image bytes.

- **Schedule JSON Prompt:** Requests structured JSON output with exact field values and schedule time mappings, used after the user confirms the extracted data.

Temperature is set to 0.1 for extraction tasks (deterministic output) and 0.0 for JSON generation (maximum consistency).

B. Automated Schedule Generation

Upon user confirmation, `ChatViewModel.performScheduling()` executes the following pipeline: (1) sends the Schedule JSON Prompt to Gemini, which outputs a JSON array with each medication’s details and computed schedule times; (2) parses the JSON response, handling edge cases such as markdown-wrapped code blocks; (3) creates `Medication` entities in Room with all extracted fields, computing `endDate` from the duration string and current date; (4) creates corresponding `Schedule` entities with appropriate hour, minute, and day-of-week values; and (5) enqueues `PeriodicWorkRequest` tasks for each schedule entry.

Default reminder times are mapped from dose notation: morning doses at 08:00, noon doses at 13:00, and night doses at 21:00. Users can adjust these times later through either the UI or the chat interface.

C. Notification and Reminder System

The reminder system is built on Android WorkManager, which guarantees execution even under battery optimization constraints, after app termination, and following device re-boots.

ReminderScheduler calculates the exact initial delay (target time minus current time; if the target has passed today, one day is added) and creates `PeriodicWorkRequest` instances with 24-hour intervals. Each request uses a unique work name per (medication, schedule) pair to prevent duplicates.

ReminderWorker creates an “unmarked” `DoseHistory` entry and displays a system notification with the medication name and dosage, including two action buttons: “✓ Taken” and “✗ Missed.”

ReminderActionReceiver handles button taps via `BroadcastReceiver`, updating the `DoseHistory` status directly without requiring the user to open the application.

BootReminderReceiver re-schedules all active reminders when the device reboots, ensuring no reminders are lost.

Custom notification sounds are supported via `MediaPlayer` (a workaround for Android 8+ channel-level sound restrictions), with users able to select per-medication ringtones.

D. Conversational AI and Action Block System

The chat interface enables natural-language medication management through a structured action block system. The user’s current medications and schedules are injected as a `[CURRENT_MEDICATIONS]` context block with each message, and the last 20 messages are included as conversation history for multi-turn coherence.

When a user requests a modification (e.g., “Change my Paracetamol reminder to 10 AM”), the AI outputs a structured action block:

```
[SCHEDULE_UPDATE]
medication: Paracetamol
old_time: 08:00 -> new_time: 10:00
[/SCHEDULE_UPDATE]
```

`ChatViewModel.parseAndExecuteAllActions()` uses regex parsing to extract these blocks, performs case-insensitive medication lookup in the database, executes the corresponding Room operations, cancels old `WorkManager` tasks, and reschedules new ones. Three action types are supported: `SCHEDULE_UPDATE`, `MEDICATION_UPDATE`, and `MEDICATION_DELETE`. The action blocks are stripped from the displayed message, so the user sees only the human-readable confirmation.

A critical design constraint ensures data integrity: the system prompt explicitly states that the AI must output the correct action block whenever it claims to have made a change, preventing false confirmations.

E. Adherence Tracking and Dose Backfilling

The adherence system includes a `DoseGenerationWorker` that runs on every app launch and resume, backfilling `DoseHistory` records from each medication’s start date through the current day. This ensures complete records exist even when the application was not running for extended periods.

`AdherenceViewModel` aggregates statistics with time-filtered queries (Today, This Week, This Month, All Time), providing per-medication adherence percentages computed from taken, missed, and unmarked dose counts.

V. DEVELOPMENT PROCESS AND RESULTS

A. Development Timeline

The project was developed iteratively over five weeks following an agile approach, with weekly deliverables presented to the faculty advisor. Table III summarizes the development timeline.

TABLE III
FIVE-WEEK ITERATIVE DEVELOPMENT TIMELINE

Week	Focus Area	Key Deliverables
1	Foundation & Setup	Infrastructure, Home UI, CameraX, Kotlin/Compose learning
2	Core Functionality	Room DB (4 entities, 4 DAOs), Add Medication form, List/Detail screens, Gemini API
3	AI Optimization	Three-pass pipeline, Bangla recognition, auto scheduling, camera UX
4	Notifications & Intel.	WorkManager, Adherence Tracker, multi-candidate verification, action blocks
5	UX Polish	Slide transitions, glassmorphism nav, inline editing, backfill worker

B. Codebase Metrics

The final codebase comprises approximately 40 Kotlin source files organized across six packages. Table IV lists the most significant files by size and role.

TABLE IV
KEY SOURCE FILES BY SIZE AND ROLE

File	Size	Role
<code>AddMedicationScreen</code>	43 KB	Complex form, dynamic pickers
<code>MedicationDetailScreen</code>	40 KB	Detail view, inline editing
<code>ChatScreen</code>	32 KB	Chat UI, markdown, images
<code>ChatViewModel</code>	29 KB	Rx-to-scheduling logic
<code>GeminiService</code>	25 KB	AI, prompts, JSON parsing
<code>CameraScreen</code>	22 KB	Camera, capture, gallery
<code>AdherenceScreen</code>	22 KB	Analytics dashboard

The project repository contains 37 commits across multiple branches (main, individual feature branches) with 5 merged pull requests, demonstrating continuous team collaboration throughout the semester.

C. Implemented Features

All five core capabilities proposed in the project proposal were successfully implemented.

- AI Prescription Scanning:** Functional extraction from both handwritten and printed prescriptions, supporting bilingual Bangla/English content and Bangladesh-specific dosage notation. Google Search grounding provides real-time medicine name verification.
- Automated Schedule Generation:** One-tap scheduling from confirmed prescription data, creating medication entries, schedule times, and `WorkManager` reminders automatically. This capability remains unique to `RxAide` among all reviewed applications.
- Reliable Medication Reminders:** `WorkManager`-based notifications persisting across app termination and device reboots, with in-notification “Taken” and “Missed” action buttons and custom per-medication notification sounds.
- Dose Adherence Tracking:** Comprehensive dashboard with overall and per-medication compliance statistics, filtered by day, week, month, or all time. Dose backfilling ensures complete records regardless of app usage patterns.
- Conversational AI Assistant:** Context-aware chat interface with the user’s medication schedule injected as context. The action block system enables real database modifications through natural language without requiring manual UI navigation.

D. Additional Features

Beyond the five core capabilities, the following features were implemented during the polish phase: dark mode toggle with custom Material 3 theme; pull-to-refresh data synchronization across all screens; inline field editing on the Medication Detail screen via tap-to-edit popups; full-screen

image preview with pinch-to-zoom and double-tap-to-zoom for scanned prescriptions in chat; custom medication form icons (tablet, capsule, cream tube, nasal spray) as vector drawables; glassmorphism effect on the bottom navigation bar; and directional slide transitions matching the bottom navigation tab order.

E. Challenges and Solutions

Several significant technical challenges were encountered and resolved during development.

- 1) **Scheduling Confirmation Flow:** The AI initially claimed to have created medication schedules without actually doing so. The fix involved scanning all chat history for unscheduled summaries, re-displaying confirmation buttons after correction conversations, adding a safety net for auto-scheduling, and updating the system prompt to clarify that scheduling is app-handled.
- 2) **Prompt Engineering Iteration:** Ensuring AI output fields exactly matched the app's `Medication` entity required iterative prompt refinement over multiple weeks, with several revisions to eliminate extraction inconsistencies.
- 3) **Dependency Conflicts:** Integrating the Google Gen AI SDK produced `META-INF` duplicate conflicts and JDK toolchain mismatches. Kotlin metadata version conflicts from Material 3 upgrades required careful dependency alignment.

VI. LIMITATIONS AND FUTURE WORK

A. Current Limitations

- 1) **Internet Dependency for AI Features:** Prescription scanning and the conversational AI assistant require an active internet connection to communicate with the Gemini API. Core functionality (reminders, dose tracking, medication management) operates fully offline.
- 2) **Extraction Accuracy Variability:** While the three-pass pipeline with multi-candidate verification significantly improves extraction reliability, extremely poor handwriting or severely degraded prescription images can still produce errors. The system mitigates this through user confirmation before scheduling and uncertainty flagging.
- 3) **Database Migration:** Room currently uses destructive migration fallback during schema changes, which is acceptable for an academic prototype but would require proper versioned migration scripts for production deployment.
- 4) **Single-User Design:** The current implementation supports a single user per device without authentication. Multi-user support and cloud synchronization are not yet implemented.

B. Future Work

Several enhancements are identified for future development: drug interaction warnings integrating a pharmaceutical database; caregiver support enabling shared adherence monitoring by family members or healthcare providers; on-device

OCR fallback using ML Kit as a backup when internet connectivity is unavailable; health measurement tracking for blood pressure, blood glucose, and other vitals alongside medication adherence; cloud synchronization for optional backup and cross-device data access; and multilingual expansion beyond Bangla/English to additional regional prescription conventions.

VII. USE OF AI TOOLS

In accordance with the course requirements for transparency, the following AI tools were used throughout the project:

- **ChatGPT (OpenAI):** Research assistance, debugging guidance, and conceptual discussions during the early design phase.
- **Gemini (Google):** Both as the core API powering RxAide's prescription scanning and chat features, and as a development assistant for Kotlin/Compose-related questions.
- **Claude (Anthropic):** Code review, debugging complex interaction flows, and assistance with prompt engineering refinement.
- **DeepSeek:** Research on specific technical implementation patterns.
- **Perplexity:** Literature search and reference verification for the literature review and project documentation.

All generated code was reviewed, understood, and adapted by team members before integration. The architectural decisions, system design, and prompt engineering approaches were developed by the team based on insights gathered from these tools and independent research.

VIII. AVAILABILITY

The complete source code for RxAide is publicly available on GitHub [24]. The repository contains all Kotlin source files, Gradle build scripts, and project documentation. A pre-built APK for direct installation on Android 8.0+ devices (API level 26 and above) is provided as the v1.0.0 release at:

<https://github.com/Aminul-Islam7/rxaide-cse299/releases/tag/v1.0.0>

To build from source, clone the repository and add a valid Google Gemini API key to `local.properties` as `GEMINI_API_KEY=<key>`, then run `./gradlew assembleDebug` using Android Studio (latest stable) with the Android SDK targeting API 36 (minimum API 26).

IX. CONCLUSION

This paper presented RxAide, an AI-powered medication reminder and management assistant that addresses three critical gaps in existing medication management applications: the Integration Gap, the Accessibility Gap, and the Intelligence Gap. Through a novel three-pass extraction pipeline with multi-candidate medicine name verification, automated schedule generation from prescription data, and a context-aware conversational AI assistant with structured action block

execution, RxAide provides a unified, end-to-end medication management experience that no existing application offers.

The system was built as a native Android application using Kotlin, Jetpack Compose, Room database, WorkManager, and the Google Gemini API, following the MVVM architecture pattern. The carefully engineered prompt architecture supports bilingual Bangla/English prescriptions and Bangladesh-specific dosage notation, making the application particularly suited for use in Bangladesh where handwritten prescriptions remain the norm.

All five core capabilities proposed in the project proposal—AI prescription scanning, automated schedule generation, reliable reminders, dose adherence tracking, and conversational AI assistance—were successfully implemented and demonstrated. The automated schedule generation capability, which converts AI-extracted prescription data into ready-to-use medication schedules with a single tap, represents a novel contribution absent from every existing application reviewed.

RxAide demonstrates that modern multimodal AI capabilities, combined with thoughtful prompt engineering and robust mobile architecture, can significantly reduce the barriers to medication adherence—particularly for elderly patients and those managing complex polypharmacy regimens who are most at risk of non-adherence.

REFERENCES

- [1] World Health Organization, *Adherence to Long-Term Therapies: Evidence for Action*. Geneva, Switzerland: World Health Organization, 2003. [Online]. Available: <https://iris.who.int/handle/10665/42682>. Accessed: May 3, 2026.
- [2] L. G. Osterberg and T. Blaschke, "Adherence to medication," *New England Journal of Medicine*, vol. 353, no. 5, pp. 487–497, Aug. 2005, doi: 10.1056/NEJMr050100.
- [3] H. B. Bosworth *et al.*, "Medication adherence: A call for action," *American Heart Journal*, vol. 162, no. 3, pp. 412–424, Sep. 2011, doi: 10.1016/j.ahj.2011.06.007.
- [4] S.-Q. Lee *et al.*, "Reasons for primary medication nonadherence: A systematic review and metric analysis," *Journal of Managed Care & Specialty Pharmacy*, vol. 24, no. 8, pp. 778–794, 2018, doi: 10.18553/jmcp.2018.24.8.778.
- [5] S. K. Kim *et al.*, "Effectiveness of mobile health intervention in medication adherence: A systematic review and meta-analysis," *Journal of Medical Systems*, vol. 49, no. 1, art. 13, Jan. 2025, doi: 10.1007/s10916-024-02135-2.
- [6] V. Lanke, K. Trimm, B. Habib, and R. Tamblyn, "Evaluating the effectiveness of mobile apps on medication adherence for chronic conditions: Systematic review and meta-analysis," *Journal of Medical Internet Research*, vol. 27, art. e60822, Jul. 2025, doi: 10.2196/60822.
- [7] Medisafe, "Medisafe Pill & Med Reminder," Google Play, 2025. [Online]. Available: <https://play.google.com/store/apps/details?id=com.medisafe.android.client>. Accessed: May 3, 2026.
- [8] smartpatient GmbH, "Meds & Pill Reminder MyTherapy," Google Play, 2025. [Online]. Available: <https://play.google.com/store/apps/details?id=eu.smartpatient.mytherapy>. Accessed: May 3, 2026.
- [9] SmartAppsFactory, "Smart Meds Reminder," Google Play, 2025. [Online]. Available: <https://play.google.com/store/apps/details?id=com.smartappsfactory.smartmedsreminder>. Accessed: May 3, 2026.
- [10] MS Production, "Prescription Reader AI," Google Play, 2025. [Online]. Available: <https://play.google.com/store/apps/details?id=com.msproduction.prescriptionreaderai>. Accessed: May 3, 2026.
- [11] AUTOOM STUDIO, "AI Prescription Saathi," Google Play, 2026. [Online]. Available: <https://play.google.com/store/apps/details?id=com.ausomemgr.prescription>. Accessed: May 3, 2026.
- [12] CareZone, "CareZone Health App." [Online]. Available: <https://www.carezone.com>. Accessed: May 3, 2026.
- [13] DoseMed, "DoseMed: AI-Powered Medication Reminder & Tracker (Privacy-First)." [Online]. Available: <https://dosemedapp.com>. Accessed: May 3, 2026.
- [14] Google AI, "Gemini API documentation," *Gemini API Docs*, 2025. [Online]. Available: <https://ai.google.dev/gemini-api/docs>. Accessed: May 3, 2026.
- [15] K. Saab *et al.*, "Capabilities of Gemini models in medicine," *arXiv preprint arXiv:2404.18416*, Apr. 2024. [Online]. Available: <https://arxiv.org/abs/2404.18416>. Accessed: May 3, 2026.
- [16] H. Fernando *et al.*, "Handwritten prescription recognition using VGG based architecture with Bi-LSTM," in *Proc. IEEE Int. Conf. Information Technology Research (ICITR)*, 2024, doi: 10.1109/ICITR61062.2024.10550875.
- [17] U. Ali *et al.*, "Leveraging deep learning with multi-head attention for accurate extraction of medicine from handwritten prescriptions," *arXiv preprint arXiv:2412.18199*, Dec. 2024. [Online]. Available: <https://arxiv.org/abs/2412.18199>. Accessed: May 3, 2026.
- [18] Android Developers, "Guide to app architecture," Android Developers, Apr. 13, 2026. [Online]. Available: <https://developer.android.com/topic/architecture>. Accessed: May 3, 2026.
- [19] Android Developers, "Get started with Jetpack Compose," Android Developers, 2025. [Online]. Available: <https://developer.android.com/develop/ui/compose/documentation>. Accessed: May 3, 2026.
- [20] Android Developers, "Save data in a local database using Room," Android Developers, Mar. 4, 2026. [Online]. Available: <https://developer.android.com/training/data-storage/room>. Accessed: May 3, 2026.
- [21] Android Developers, "Task scheduling | Background work," Android Developers. [Online]. Available: <https://developer.android.com/develop/background-work/background-tasks/persistent>. Accessed: May 3, 2026.
- [22] V. Kini and P. M. Ho, "Interventions to improve medication adherence: A review," *JAMA*, vol. 320, no. 23, pp. 2461–2473, Dec. 2018, doi: 10.1001/jama.2018.19271.
- [23] L. Flees and S. Slovenski, "Further enhancing our digital health & wellness capabilities," Walmart Corporate, Jun. 15, 2020. [Online]. Available: <https://corporate.walmart.com/news/2020/06/15/further-enhancing-our-digital-health-wellness-capabilities>. Accessed: May 3, 2026.
- [24] A. Islam, M. S. R. Rabbi, and T. Mojumdar, "RxAide: AI-Powered Medication Reminder and Management Assistant," GitHub, Apr. 30, 2026. [Online]. Available: <https://github.com/Aminul-Islam7/rxaide-cse299>. Accessed: May 3, 2026.